

CSS

Flexbox e CSS Grid: construindo layouts de verdade



Por que Flexbox?

O problema antes do Flexbox

- Alinhar elementos lado a lado exigia float ou display: inline-block
- Centralizar verticalmente era famosa dor de cabeça (truques com position + transform)
- Distribuir espaço igualmente entre itens exigia cálculos manuais de margin
- Qualquer mudança de conteúdo quebrava o alinhamento

O que o Flexbox resolve

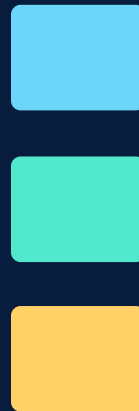
- Alinha itens em uma única linha ou coluna (layout 1D)
- Centraliza vertical e horizontalmente com 2 propriedades
- Distribui espaço automaticamente entre os itens
- Reflow automático quando o conteúdo muda de tamanho
- Ideal para: barras de navegação, cards em linha, formulários, centralização

Ativando o Flexbox: display e direction

```
.container {  
  display: flex;  
  flex-direction: row;  
  /* ou: column */  
}  
  
<div class="container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
</div>
```



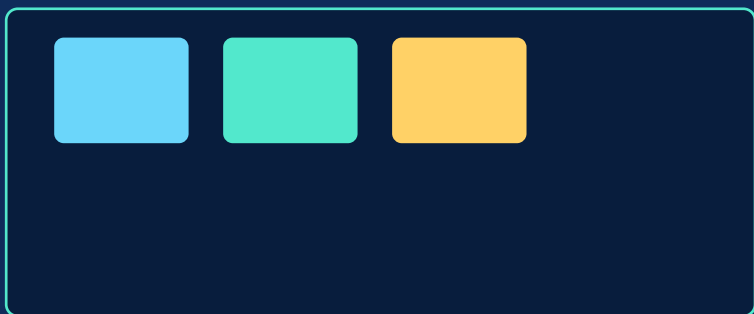
flex-direction: row (padrão)



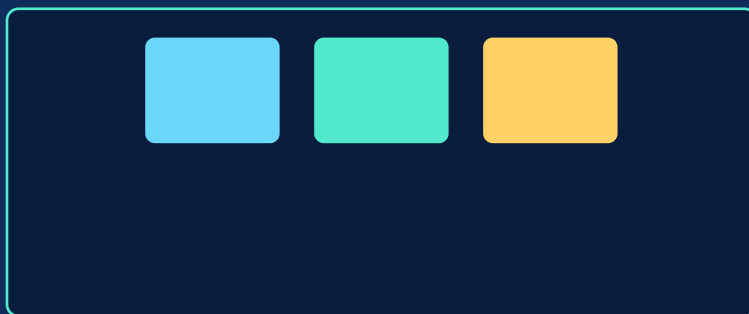
flex-direction: column

justify-content: eixo principal

Controla como os itens se distribuem ao longo da linha (eixo principal).



flex-start



center



space-between

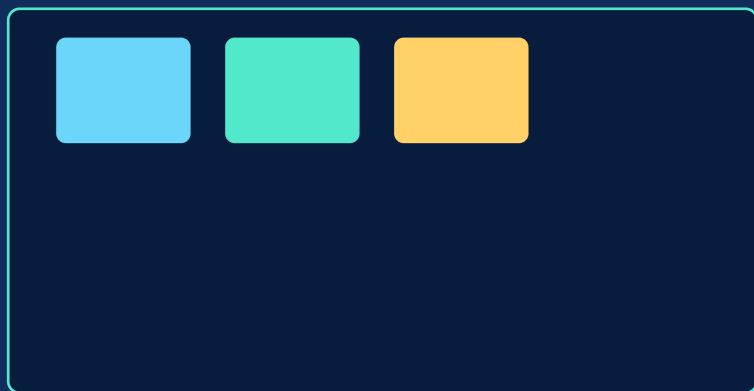


space-around

- space-between: espaço só entre os itens (pontas coladas nas bordas)
- space-around: espaço igual ao redor de cada item (bordas com metade do espaço)
- Aplicação real: barra de navegação com logo à esquerda e links à direita usa space-between

align-items: eixo secundário

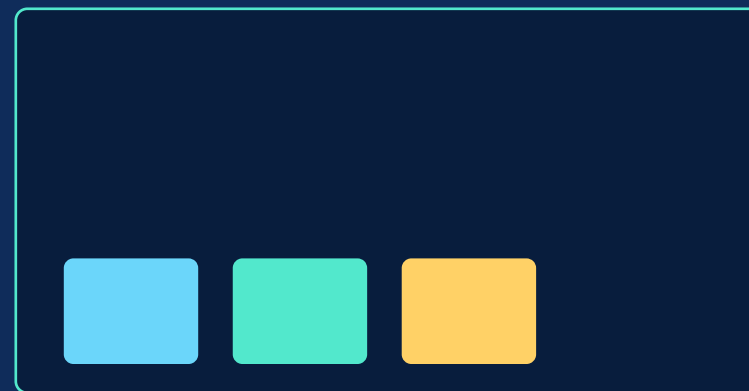
Controla como os itens se alinham na direção perpendicular ao eixo principal.



flex-start



center



flex-end

Combo mais usado do curso

- `display: flex; justify-content: center; align-items: center;` — centraliza qualquer coisa, em qualquer tamanho de tela
- Esse combo substitui truques antigos de centralização vertical e deve ser memorizado

flex-wrap e gap

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 16px;  
}  
  
/* sem wrap: itens espremem */  
/* ou saem da tela */  
  
/* com wrap: itens quebram */  
/* linha quando não cabem */
```

Por que isso é essencial

- flex-wrap: wrap permite que os itens quebrem para a próxima linha em telas menores
- Sem wrap, o padrão (nowrap) força tudo em uma linha só — itens espremem ou vazam da tela
- gap cria espaço entre os itens sem precisar de margin manual em cada um
- gap funciona tanto em Flexbox quanto em Grid — uma propriedade, dois contextos
- Aplicação real: cards de produto que reorganizam sozinhos ao redimensionar a janela

Propriedades dos itens: grow, shrink, basis

```
.item {  
  flex-basis: 200px;  
  flex-grow: 1;  
  flex-shrink: 1;  
}  
  
/* atalho equivalente */  
.item {  
  flex: 1 1 200px;  
}
```

O que cada um faz

- flex-basis: tamanho inicial do item antes de crescer ou encolher
- flex-grow: o quanto o item cresce para preencher espaço sobrando (0 = não cresce)
- flex-shrink: o quanto o item encolhe quando falta espaço (0 = não encolhe)
- flex: grow shrink basis é o atalho mais usado na prática
- Ex.: flex: 1 faz 3 itens dividirem o espaço em partes iguais, automaticamente

Navbar com Flexbox

```
nav {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 16px 32px;  
}  
  
nav ul {  
  display: flex;  
  gap: 24px;  
  list-style: none;  
}
```

O que está acontecendo

- O <nav> vira um flex container: logo de um lado, menu do outro
- space-between empurra os dois grupos para as extremidades
- align-items: center alinha tudo verticalmente, mesmo com alturas diferentes
- A do menu também vira flex container, com gap entre os links
- Este é o padrão usado em praticamente todo site profissional

Cards em linha, com quebra automática

```
.cards {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
}  
  
.card {  
  flex: 1 1 240px;  
  background: var(--cor-fundo);  
  padding: 20px;  
  border-radius: 10px;  
}
```

Por que esse padrão funciona tão bem

- flex: 1 1 240px diz: comece com 240px, mas cresça e encolha conforme o espaço
- flex-wrap: wrap garante que os cards quebrem linha em telas estreitas
- gap cuida do espaçamento — sem margin manual em cada card
- Resultado: uma galeria de cards responsiva, sem nenhuma media query
- Aplicação direta: cards de depoimentos do exercício do café

O que é CSS Grid?

Flexbox x Grid

- Flexbox organiza em uma direção por vez (1D): linha OU coluna
- Grid organiza linhas E colunas ao mesmo tempo (2D)
- Use Flexbox para: navbars, grupos de botões, cards em linha
- Use Grid para: layouts de página inteira, galerias, dashboards
- Os dois convivem no mesmo projeto — não competem entre si

```
.grid {  
  display: grid;  
  grid-template-columns:  
    1fr 1fr 1fr;  
  gap: 20px;  
}  
  
/* cria 3 colunas de  
   largura igual */
```

grid-template-columns e a unidade fr

```
.grid-a {  
  grid-template-columns:  
    1fr 1fr 1fr;  
}  
/* 3 colunas iguais */  
  
.grid-b {  
  grid-template-columns:  
    250px 1fr;  
}  
/* sidebar fixa + conteúdo  
   que ocupa o resto */
```

Entendendo fr (fraction)

- fr representa uma fração do espaço disponível no container
- 1fr 1fr 1fr divide em 3 partes iguais
- 2fr 1fr faz a 1ª coluna ocupar o dobro da 2ª
- Pode misturar unidades fixas (px) com fr — muito comum em layouts com sidebar
- Nunca precisa calcular porcentagem manualmente de novo

repeat(), auto-fit e minmax(): grid responsivo sem media query

```
.galeria {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit,  
      minmax(220px, 1fr));  
  gap: 20px;  
}
```

- Esta é, sozinha, a linha de CSS mais reaproveitada em projetos reais

Lendo a fórmula por partes

- repeat(N, valor): repete o valor N vezes sem precisar copiar e colar
- auto-fit: calcula automaticamente quantas colunas cabem na largura disponível
- minmax(220px, 1fr): cada coluna nunca fica menor que 220px, mas cresce até 1fr
- Resultado: colunas que aparecem e somem sozinhas ao redimensionar a tela
- Sem nenhuma media query — o Grid resolve isso sozinho

grid-template-areas: nomeando o layout

```
.pagina {  
  display: grid;  
  grid-template-areas:  
    "cabecalho cabecalho"  
    "menu conteudo"  
    "rodape rodape";  
  grid-template-columns: 200px 1fr;  
}
```

```
header { grid-area: cabecalho; }  
nav     { grid-area: menu; }  
main    { grid-area: conteudo; }  
footer  { grid-area: rodape; }
```

Por que isso é poderoso

- Cada string representa uma linha do layout — visualmente parecido com o resultado final
- Nomeia regiões (cabecalho, menu, conteudo, rodape) em vez de contar linhas/colunas
- Cada elemento HTML só precisa saber seu grid-area, sem se preocupar com posição numérica
- Ótimo para layouts de página inteira: cabeçalho + sidebar + conteúdo + rodapé

Galeria de produtos responsiva com Grid

```
.galeria-produtos {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit,  
      minmax(200px, 1fr));  
  gap: 24px;  
}  
  
.produto {  
  background: var(--cor-fundo);  
  border-radius: 10px;  
  padding: 16px;  
  text-align: center;  
}
```

Onde aplicar nos seus projetos

- Cardápio do exercício do café: cada item vira um `.produto` dentro de `.galeria-produto`
- Em telas largas: várias colunas; em telas estreitas: 1 coluna, sozinho
- Nenhuma media query foi usada — só `grid-template-columns` bem escrito

Desafios de hoje: Flexbox e Grid

Como funciona

- Cada desafio parte de um modelo (HTML/CSS já pronto, sem Flexbox/Grid aplicado)
- A tarefa é sempre pequena: adicionar de 3 a 6 linhas de CSS para resolver um problema específico
- Pode ser feito em dupla — um lê o desafio em voz alta, o outro digita

Desafios de Flexbox

- Alinhar uma barra de navegação (logo + menu)
- Centralizar um card de aviso na tela inteira
- Fazer uma lista de tags quebrar linha sozinha

Desafios de Grid

- Transformar uma lista de cards em galeria responsiva
- Criar um rodapé com 3 colunas de informação
- Montar um layout de página com áreas nomeadas